



HI-TECH C Compiler for PIC10/12/16 MCUs Version 9.70 Release Notes

Copyright (C) 2009 Microchip Technology Inc.
All Rights Reserved. Printed in Australia.
Produced on: October 16, 2009

Australian Design Centre
45 Colebard Street West
Acacia Ridge QLD 4110
Australia

web: <http://www.htsoft.com>

**THIS FILE CONTAINS IMPORTANT INFORMATION RELATING TO THIS
COMPILER. PLEASE READ IT BEFORE RUNNING THIS SOFTWARE.**

Chapter 1

Introduction

1.1 Description

This version of the compiler is a significant release which principally implements two important changes, being the second generation of our Omniscient Code Generation (OCG) technology, and three operating modes into the one compiler package.

OCG is a framework the HI-TECH branded compilers use to compile projects consisting of multiple source files. It is a novel approach that generates code for all parts of the C program — user source and libraries — in the one step. This allows the compiler to derive complete knowledge of the C program and in turn allows the compiler to perform tasks that would normally require the use of non-standard keywords.

This framework has been improved and extended in this version of the compiler. The changes allow several compiler limitations to be lifted, making the compiler more versatile and reducing the workload for programmers. In addition, more optimizations have been implemented that take advantage of the knowledge derived from this framework.

This compiler release replaces both the previous HI-TECH C Compiler for PIC10/12/16 MCUs (formally the PRO compiler) and the HI-TECH PICC STD Compiler for the Microchip PICmicro. The PICC STD compiler will no longer be developed but will continue as an operating mode of this and subsequent compiler releases.

As with all releases, support for the latest devices has been added, and bugs reported in previous versions have been addressed in this version.

1.2 Updates and Feedback

Microchip welcomes bug reports, suggestions or comments regarding this compiler version. Please direct any bug reports or feature requests via email to ([support](#)).

1.3 Previous Versions

The previous version of HI-TECH C Compiler for PIC10/12/16 MCUs was 9.65PL1, released in June 2009. The previous HI-TECH PICC STD Compiler for the Microchip PICmicro was 9.60PL3, released in January 2009.

Chapter 2

New Features

2.1 General

New devices supported (9.70) The following devices are now supported by the compiler: 12F520, 12F1822, 12LF1822, 12F617, 16F1823, 16LF1823, 16F1938, 16LF1938, 16F1939, 16LF1939, 16F1946, 16LF1946, 16F1947, 16LF1947, 16F707 and 16LF707.

Standard operating mode (9.70) This compiler can operate in one of three mode: PRO, Standard or Lite. PRO and Standard modes require a license, but may be evaluated for 45 days free of charge. Lite mode is always available. The difference in operating mode is purely one of output code size — all source code features and code generation options are implemented in all modes. Providing the license allows, the compiler may be switched from one mode to another on a build-by-build basis.

2.2 Code Generator

Absolute addressed const objects (9.70) Both initialized and uninitialized `const`-qualified objects can now be placed at an absolute address. This feature allows users to position data in program memory at a specific locations with out the need for assembly code. Examples:

```
const unsigned short foo[] @ 0x100 = { 0xADDE, 0xEFBE, 0xEDFE };  
const int bar @ 0x200 = 0x0DD0;
```

Functions larger than a page (9.70) The compiler can now compile functions whose assembly output is larger than a ROM page size. This is the first mid-range compiler that has this limitation lifted. No source code changes are required for this to take place. This feature is not available for baseline devices.

Improved memory allocation for local objects (9.70) The memory allocation strategy for local objects: autos, parameters and temporary variables, has been improved. Previously a compiled stack was formed from the auto-parameter blocks of each function in the program and was allocated one contiguous range of memory that had to be located within one RAM bank. This limitation has been lifted. The compiled stack can now consist of smaller stacks, each located in separate RAM banks. The RAM bank chosen for a function's auto variables may be different to that used by its parameters, and indeed the auto variables associated with one function may even be spread over multiple RAM banks, if required. The parameters and temporary variables associated with a function will only be allocated to one RAM bank, but those of other functions may use different banks.

RAM objects larger than one bank (9.70) When the target device is an enhanced mid-range PIC device, the source code may define RAM-based objects (arrays and structures) that are larger than one bank in size. Such objects will be accessed indirectly using the linear data memory. This feature is not available when a baseline or mid-range target device is selected.

More than one bank of initialized objects (9.70) Previously the total size of initialized objects defined by a program must be less than the size of a RAM bank. This limitation has been lifted. Initialized objects can reside in more than one bank, and the automatically generated runtime startup code can ensure that all areas are correctly initialized.

Psect pragma reinstated (9.70) The `#pragma psect` directive, which was originally present on STD compilers, was initially not available on PRO compilers. This directive has now been reinstated and operates as it did previously. Although most objects can now be placed at specific locations by making them absolute, the psect pragma permits more control over how objects are positioned by allowing use of the many features in the linker.

2.3 MPLAB Plugin

Plugin version 1.35 (9.70) The Windows version of this release features a new version of the HI-TECH Universal Toolsuite MPLAB plugin, viz. version 1.35. Only this version of the plugin is compatible with HI-TECH C Compiler for PIC10/12/16 MCUs 9.70. The plugin is installed automatically when the compiler is installed. The changes to the plugin reflect changes to the driver options in 9.70.

Chapter 3

Changes

3.1 Driver

-apbank option (9.70) This option is no longer applicable. If used the driver will ignore it.

Linker option overrides (9.70) The operation of the `-L-` driver option, which passes options directly to the linker, has changed. If this option is being used to add additional linker options, then there is no change required in the way it is used. If this option is being used to replace default linker options then there may be changes required to the driver options. Previously if the string up to the first equal sign matched the *initial* part of a default linker option, that entire linker option would have been removed and what follows the `-L-` would be added as a new option. Now a search is now made for the string (up to the first equal sign in the `-L-` option) and each and every psect or class name in the default linker options. If a match is found, that psect/class specification and the remaining part of the linker option is removed and the new option added.

Special pointer targets (9.70) Pointers and integers are not interchangeable. Assigning an integer constant to a pointer will generate a warning to this effect. There is no information in the integer constant relating to the type, size or memory location of the destination. There is a very good chance of code failure if pointers are assigned integer addresses and dereferenced, particularly for devices like PIC devices which have more than one address space. For information about this issue and ways of dealing with it, refer to *Special Pointer Targets* in the user manual.

3.2 Code Generator

Assembly names of auto symbols (9.70) Previously `auto` variables associated with a function could be accessed in assembly code either by a linker-defined symbol of the form `??_funcName+offset` (where *funcName*

is the name of the function which defines the auto, and *offset* is a numerical offset from this symbol) or by a code-generator-defined symbol of the form `funcName_varName`. The first form can no longer be used. Note also that in the latter form, the *underscore* character has been replaced with an @ symbol, so for example an auto variable called `buf` defined in the `main` function could be accessed in assembly code using the symbol `main@buf`. Parameters and temporary variables can be accessed using either form.

Assembly names of static functions and variables (9.70) Previously static symbols had assembly domain symbols that were based on an “F” symbol, e.g. `_add_F4035`. Now all assembler domain static symbols follow a similar convention to that for other locals. The assembly name of static variables are of the form `funcName@varName` where *funcName* is the name of the function that defines the symbol. If the variable is not defined inside a function, the symbol has the form: `fileName@varName`, where *fileName* is the name of the source file that contains the definition. If there are two static functions with the same name and each contain a static variable of the same name, then the variables will be accessible using the form: `fileName@funcName@varName`.

#pragma interrupt_level (9.70) The automatic duplication of functions can be inhibited by preceeding a function definition with the `interrupt_level` pragma. In addition to preventing the function from being duplicated, all functions that it calls, and all functions called by those calls, etc... will also be prevented from being duplicated.

3.3 Library

Floating-point arithmetic functions (9.70) The compiler library functions which provide 24- and 32-bit arithmetic routines have been updated so that they require less memory and have faster execution times.

Chapter 4

Limitations

4.1 Code Generator

eeeprom qualifier (9.70) This qualifier is not available and its use will result in undefined behaviour. This qualifier is planned for a subsequent release and information in the manual regarding this qualifier should be disregarded until that time.

option and tris (9.70) For baseline devices, the OPTION and TRIS registers must be read/written as a byte. Reading or writing individual bits is not supported.

PIC17 support (9.70) PIC 17 devices (for example, 17C756) is not supported by this compiler.

fast32 floats (9.70) The option to select the fast 32-bit float or double library for PIC17 devices that was included in the PICC STD compiler is no longer available.

4.2 Libraries

Peripheral routines (9.70) The flash read/write routines/macro included with this compiler are functional only for devices that can write one word at a time (e.g. 16F877).

